



Department of Computer science and AI

Course: Artificial Intelligence

Instructor: Warda Aslam

Project Report

Title:

AI INTERIOR DESIGN VISUALIZER

Group Members	Reg Id
Sibghatullah Asif	230896
Noor ul Ain	230924
Mahnoor Iqbal	230910

Table of Contents

1. Introduction	3
2. Problem Statement.....	3
3. Objectives of the Project.....	4
4. Scope of the Project	4
5. Tools and Technologies Used	4
6. System Architecture.....	5
7. Dataset Description.....	9
8. Data Preprocessing	11
9. Exploratory Data Analysis	12
10. AI Models Used.....	13
11. Model Comparison	14
12. Evaluation Metrics and Results	14
13. Backend Implementation	15
14. Frontend Implementation.....	17
15. System Workflow	20
17. Testing and Discussion.....	20
17.1 Functional Testing	21
17.2 API Testing	22
18. Limitations.....	23
19. Future Enhancements	24
20. Conclusion.....	24
21. References	24

1. Introduction

Interior design is an important part of making a living space more comfortable, attractive, and useful. Many people want to redesign their rooms but they are unsure about wall colors, furniture styles, and overall room appearance. Usually, people buy paint or furniture first and later realize that the selected color or style does not match the room. This can waste both time and money.

The AI Interior Design Visualizer is developed to solve this problem. The application allows the user to upload a room image and preview possible design changes digitally. The system identifies room areas such as walls, floor, and furniture, and then applies design changes to show a redesigned version of the room. The user can compare the original image with the redesigned image in a side-by-side view.

The project uses artificial intelligence and image processing techniques. Semantic image segmentation is used to separate different parts of the room. OpenCV and NumPy are used for image handling and color processing. Flask is used for backend APIs, while React.js and Fabric.js are used to build the frontend interface and canvas-based editing system.

This project is useful because it gives users a simple way to test interior design ideas before applying them in real life. It also shows how AI can be used in practical daily-life applications.

2. Problem Statement

Many users find it difficult to imagine how their room will look after changing wall colors, furniture style, or decoration. Traditional interior design requires manual planning, designer consultation, or physical changes, which can be costly and time-consuming. Users may select furniture or paint without knowing whether it will match the existing room environment.

The main problem is that users need an easy and visual way to preview room redesigns before purchasing anything. A simple image editor is not enough because the system should understand different room areas like walls, floor, and furniture. Therefore, an AI-based solution is needed that can segment the room image and apply design changes to the correct regions.

The AI Interior Design Visualizer solves this problem by allowing users to upload a room image, segmenting the image into meaningful areas, suggesting color palettes, and generating a redesigned preview in the browser.

3. Objectives of the Project

The main objectives of this project are:

- To develop a web-based AI interior design visualizer.
- To allow users to upload room images.
- To segment room areas such as walls, floor, and furniture.
- To provide wall color changing functionality.
- To generate color palette recommendations.
- To show side-by-side before and after comparison.
- To provide canvas-based editing using Fabric.js.
- To allow users to save and share redesign outputs.
- To compare at least two segmentation/design processing models.

4. Scope of the Project

The scope of this project is limited to browser-based room redesign visualization. The system accepts an uploaded room image and generates a redesigned preview. It focuses mainly on wall recoloring, furniture/decor style preview, palette suggestion, and before/after comparison.

The system does not physically measure room dimensions and does not create 3D room models. It is a 2D image-based visualization tool. The project is suitable for students, homeowners, renters, and small interior planning use cases.

5. Tools and Technologies Used

Python

Python is used for backend development and image processing. It provides strong support for AI models, image manipulation, and scientific computing.

PyTorch

PyTorch is used for AI-based segmentation model support. In this project, segmentation is used to identify different regions of a room image.

SAM / DeepLab

Semantic segmentation is one of the core AI components of this project. SAM by Meta or DeepLab-style segmentation can be used to divide an image into meaningful regions. These regions are later used for wall recoloring and furniture/decor modification.

OpenCV

OpenCV is used for image processing tasks such as reading images, resizing, mask generation, contour detection, and region-based operations.

Flask

Flask is used to create backend APIs. The frontend communicates with Flask APIs to upload images, request palette suggestions, generate redesign previews, and save design data.

React.js

React.js is used for frontend development. It provides the user interface for image upload, before/after comparison, color picker, furniture catalog, and result display.

Fabric.js

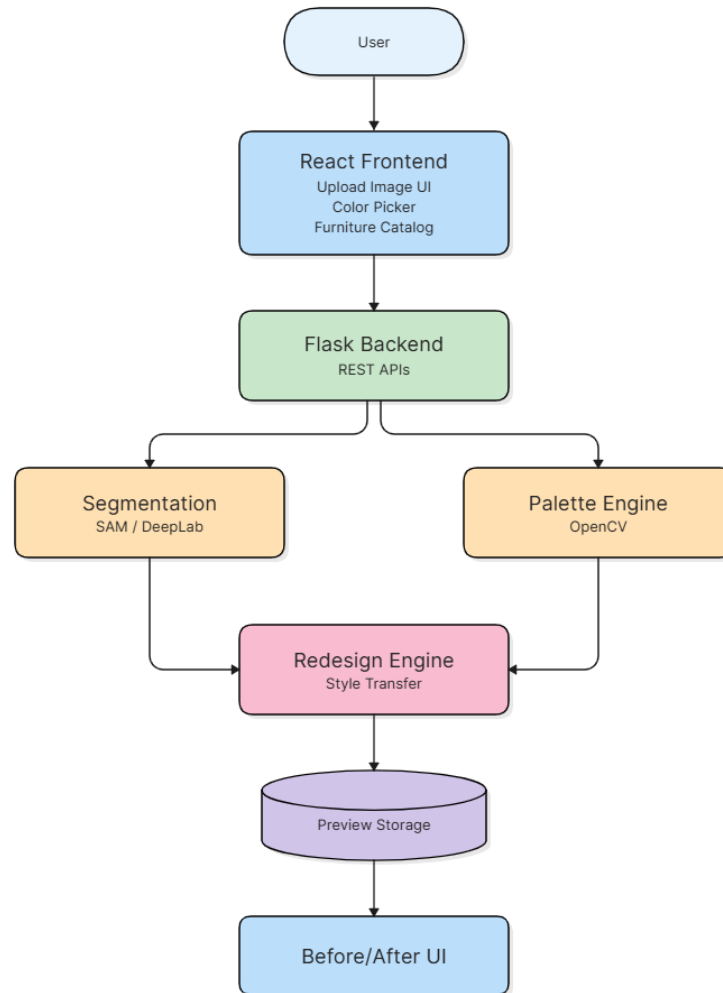
Fabric.js is used for canvas-based editing. It helps display editable layers, furniture objects, and mask regions inside the browser.

6. System Architecture

The system follows a client-server architecture. The frontend is built using React.js and communicates with the Flask backend through REST APIs. The backend processes uploaded images and returns segmentation results, palettes, and preview images.

High-Level Architecture

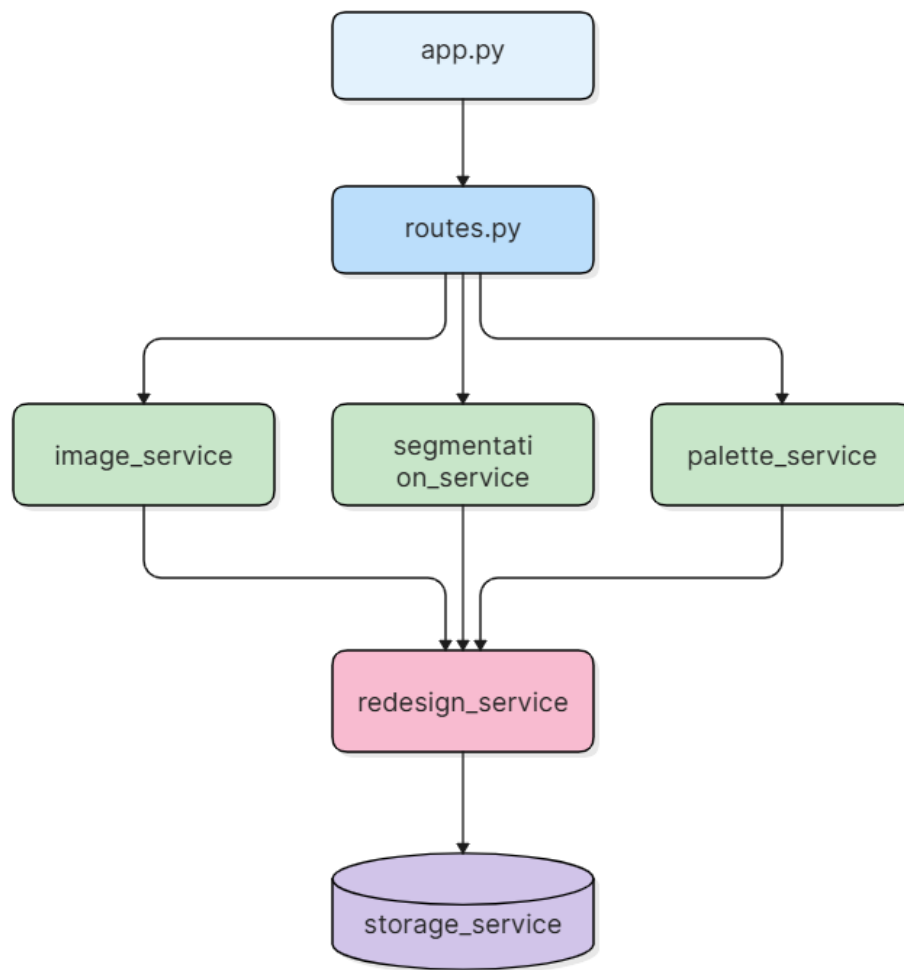
The user interacts with the React frontend. The frontend sends the uploaded image to the backend. The backend performs preprocessing, segmentation, palette extraction, and redesign generation. The final output is returned to the frontend and displayed in a before/after view.



Backend Architecture

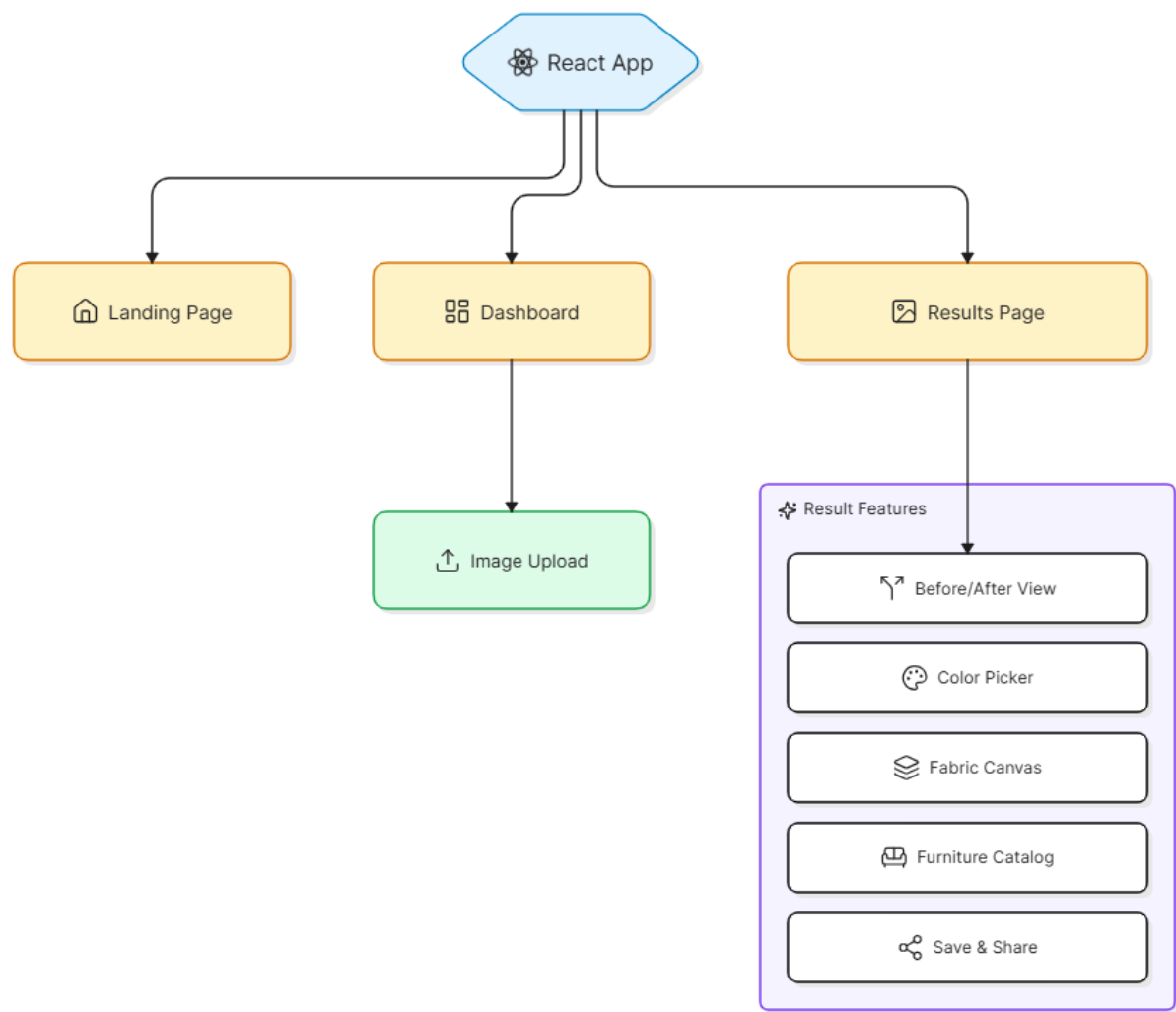
The backend is divided into multiple modules:

- `app.py` starts the Flask application.
- `routes.py` contains API routes.
- `config.py` stores paths and configuration.
- `image_service.py` handles image upload and image decoding.
- `segmentation_service.py` handles AI segmentation.
- `palette_service.py` extracts color palettes.
- `redesign_service.py` applies color/style changes.
- `storage_service.py` saves and loads design data.



Frontend Architecture

The frontend contains different pages and components. The main pages include landing page, dashboard/upload page, results page, analytics page, and about page. The frontend handles image preview, color picker, furniture catalog, canvas editing, and save/share options.



API Flow

The major API routes used in the system are:

API Route	Method	Purpose
/api/health	GET	Check backend status
/api/palette	POST	Extract room color palette
/api/redesign	POST	Generate redesigned preview

/api/segment	POST	Return segmented room regions
/api/catalog	GET	Return furniture catalog
/api/designs/save	POST	Save redesign payload
/api/designs/<id>	GET	Load saved design
/api/uploads/<filename>	GET	Serve uploaded image
/api/previews/<filename>	GET	Serve generated preview

7. Dataset Description

For this project, a small custom dataset of room images was used for testing. The dataset included different indoor room types such as bedrooms, living rooms, kitchens, and simple study rooms. The purpose of the dataset was to test whether the system can process uploaded room images, detect regions, extract colors, and generate redesigned outputs.

The dataset was not used for training a model from scratch. Instead, it was used for testing and evaluation of segmentation and redesign results. The segmentation model works on uploaded images and produces masks for room regions.

Dataset Sources

The dataset can be created using:

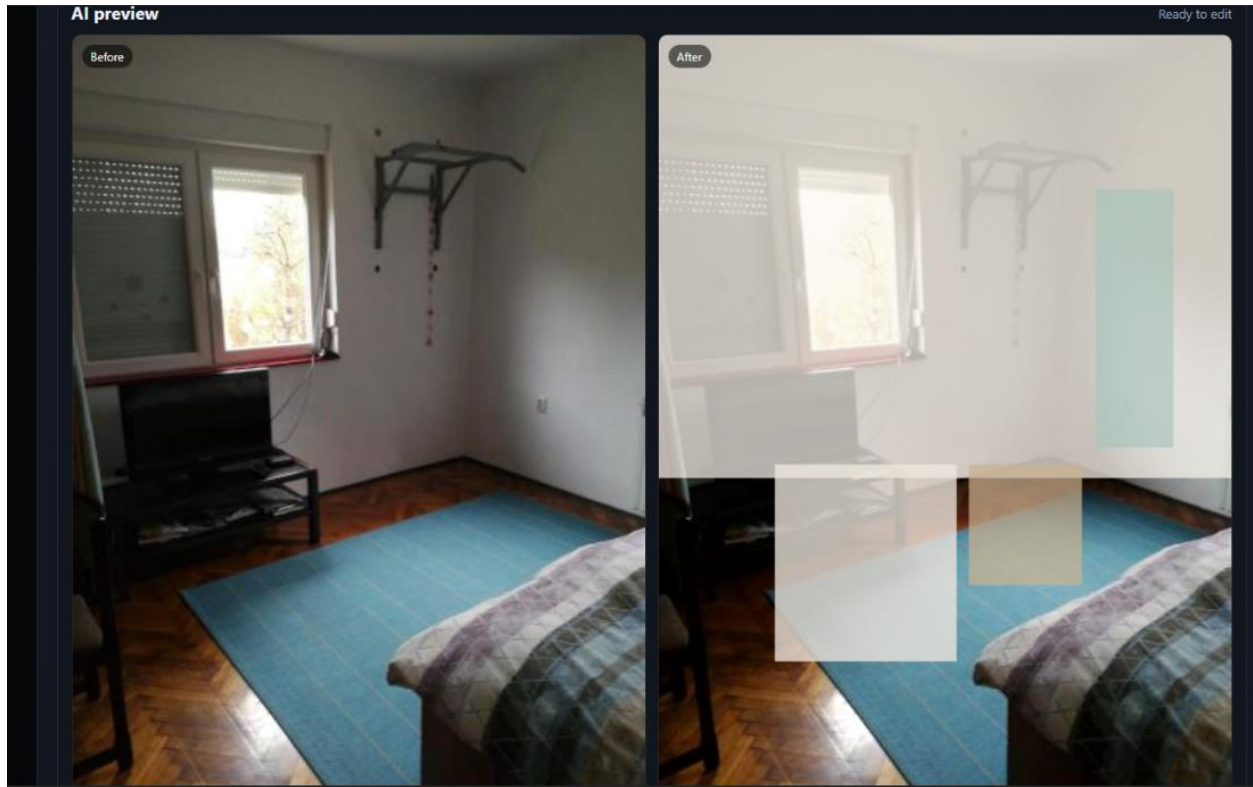
- Manually captured room photos.
- Sample room images from public image sources.
- User-uploaded test images.
- Images collected from personal room environments.

Dataset Features

Feature	Description
Image Type	Indoor room images
Format	JPG, PNG
Room Types	Bedroom, living room, kitchen, study room
Input Size	Different sizes, resized during preprocessing
Labels	Wall, floor, furniture masks generated during segmentation
Output	Redesigned preview image

Sample Dataset Images





8. Data Preprocessing

Before applying segmentation and redesign, the uploaded image goes through preprocessing. Preprocessing is important because uploaded images can have different sizes, formats, lighting conditions, and orientations.

Image Upload and Reading

The user uploads an image through the frontend. The backend receives the image through API request and reads it using image processing libraries. If the image is sent as base64 data, it is decoded first.

Image Conversion

The image is converted into RGB format. This step makes sure that the image has a consistent color format for segmentation and palette extraction.

Image Resizing

Large images can slow down processing. Therefore, images may be resized to a manageable resolution before segmentation. Resizing also helps in faster mask generation.

Noise Handling

Some images contain shadows, blur, or uneven lighting. Basic preprocessing helps reduce small noise effects. OpenCV can be used for smoothing and mask cleaning.

Mask Preparation

After segmentation, masks are converted into binary form. These masks are later used for applying color changes to walls, floor, and furniture regions.

9. Exploratory Data Analysis

Exploratory Data Analysis was performed to understand the image dataset and color patterns. Since this project deals with room images, EDA focused on image dimensions, room categories, dominant colors, brightness, and palette distribution.

Room Type Distribution

The dataset included multiple types of room images. Bedrooms and living rooms were more common because they are usually used for interior design testing.

[Insert Chart here: Bar chart of room types such as bedroom, living room, kitchen, study room]

Image Size Analysis

Images had different resolutions. Some images were small, while others were high-resolution. For better performance, large images were resized before processing.

Brightness Analysis

Brightness affects segmentation and color extraction. Very dark images may produce less accurate masks. Good lighting improves the quality of segmentation and palette recommendation.

Dominant Color Analysis

Dominant colors were extracted from room images. These colors were used to generate palette suggestions for wall and accent colors.

EDA Summary

EDA showed that indoor images have different lighting conditions, room layouts, and color themes. This makes the project challenging because the system must work on different types of uploaded room images. However, preprocessing and segmentation help improve the final output.

10. AI Models Used

This project compares two approaches for room segmentation and redesign processing.

Model 1: SAM-Based Segmentation

SAM by Meta is used for segmentation. It can generate masks for different objects and regions in an image. In this project, SAM helps identify possible wall, floor, and furniture regions. These masks are then used for color editing and redesign preview.

SAM is useful because it can work on general images and does not require training from scratch for this project. It is suitable for room image segmentation because it can separate visible regions and objects.

Model 2: DeepLab / Classical Segmentation Baseline

A second approach was used as a comparison baseline. This approach can be DeepLab-style semantic segmentation or a fallback OpenCV-based region segmentation method. The baseline helps compare performance, speed, and output quality.

In the baseline method, image regions are segmented using simpler logic or pretrained segmentation behavior. It is faster but may be less accurate compared to SAM-based segmentation.

Why Two Models Were Compared

The purpose of comparing two models is to understand which approach gives better segmentation quality for interior design images. SAM gives better region separation, while the

baseline method is faster and simpler. The comparison helps choose the better model for final application use.

11. Model Comparison

The two models were compared on segmentation quality, processing speed, and visual output quality.

Comparison Criteria

Criteria	Description
Segmentation Quality	How well walls, floor, and furniture are separated
Processing Time	Time required to generate masks
Output Quality	How realistic the redesigned preview looks
Ease of Integration	How easily the model works with Flask backend
Browser Usability	Whether output can be used in frontend canvas

Discussion of Comparison

The SAM-based approach produced better segmentation masks because it was able to separate different room objects and areas more clearly. It performed better in images where walls, furniture, and floor were visually distinct. However, SAM required more processing time compared to the baseline method.

The baseline method was faster but less accurate. It sometimes generated rectangular or rough masks, which did not always match the real shape of walls or furniture. For this reason, SAM-based segmentation was selected as the preferred method for final redesign generation.

12. Evaluation Metrics and Results

Evaluation was performed to check the quality of segmentation and redesign output. Since the project is image-based, both quantitative and visual evaluation were used.

Intersection over Union

Intersection over Union measures the overlap between predicted mask and expected mask. Higher IoU means better segmentation.

Formula:

$\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$

Mask Accuracy

Mask accuracy checks how correctly the model identifies wall, floor, and furniture regions.

Processing Time

Processing time measures how long the backend takes to generate segmentation and preview output.

Visual Quality Score

Visual quality was checked manually by comparing before and after images. A good output should show visible design changes without damaging the original room structure.

Evaluation Results

Test Case	Room Type	Wall Detection	Furniture Detection	Preview Quality	Remarks
1	Bedroom	Good	Medium	Good	Wall recolor visible
2	Living Room	Good	Good	Good	Furniture region detected
3	Kitchen	Medium	Medium	Average	Cabinets and walls mixed
4	Study Room	Good	Medium	Good	Clean result
5	Bedroom	Good	Good	Good	Best output

Result Summary

The system performed better on images with clear walls, visible floor, and separate furniture objects. Images with shadows, blur, or crowded furniture were more difficult to segment. The final output was acceptable for a prototype AI interior design visualizer.

13. Backend Implementation

The backend was developed using Flask. It receives requests from the frontend and performs all AI and image processing tasks.

Flask Application

The Flask app is initialized in app.py. It enables CORS so that the React frontend can communicate with the backend.

Routes

The main routes are defined in routes.py. These routes handle image upload, segmentation, palette extraction, redesign generation, furniture catalog, and saving design payloads.

Image Service

The image service handles uploaded images. It supports multipart image upload, base64 image data, and previously uploaded image URLs.

Segmentation Service

The segmentation service generates room masks. It uses AI-based segmentation to identify possible room regions. The masks are converted into vector layers so they can be used in Fabric.js canvas.

Palette Service

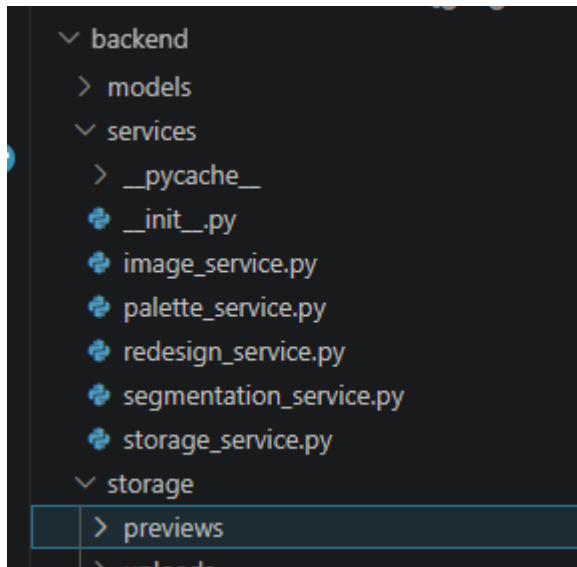
The palette service extracts dominant colors from the uploaded room image. It uses clustering and color analysis to generate useful swatches.

Redesign Service

The redesign service applies design changes to the segmented regions. For example, it changes wall color using the selected color and applies theme-based tinting to furniture and floor regions.

Storage Service

The storage service saves generated design data in JSON format. It also helps load saved designs using design IDs.

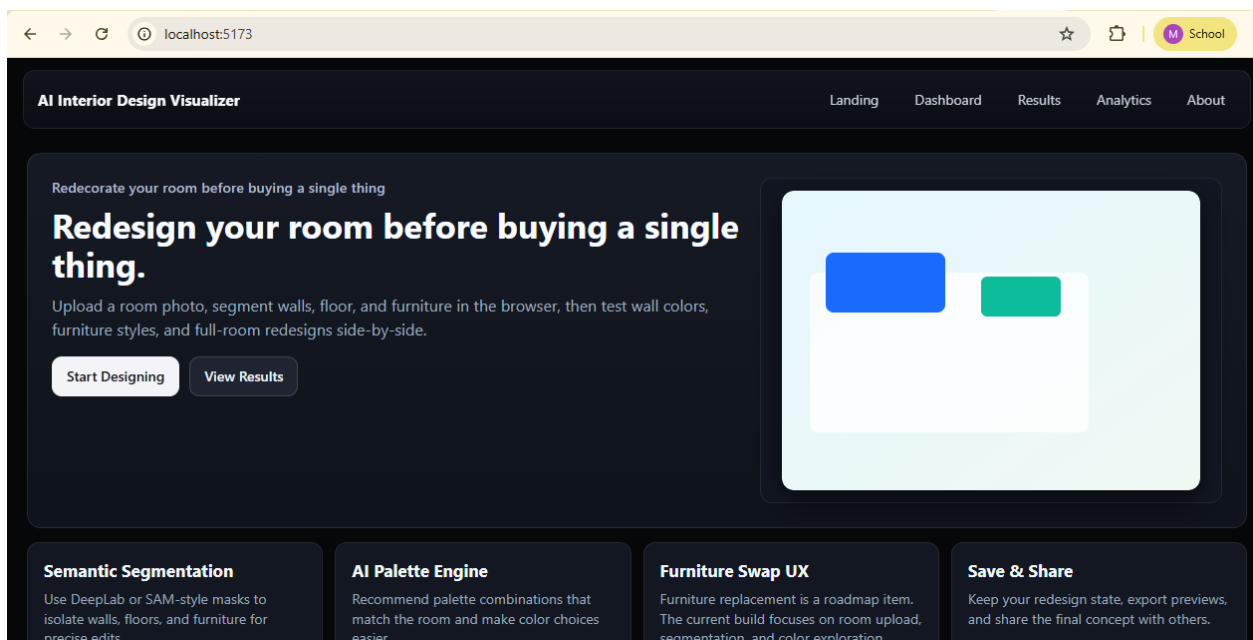


14. Frontend Implementation

The frontend was developed using React.js. It provides an interactive interface where users can upload images, choose colors, view results, and save/share designs.

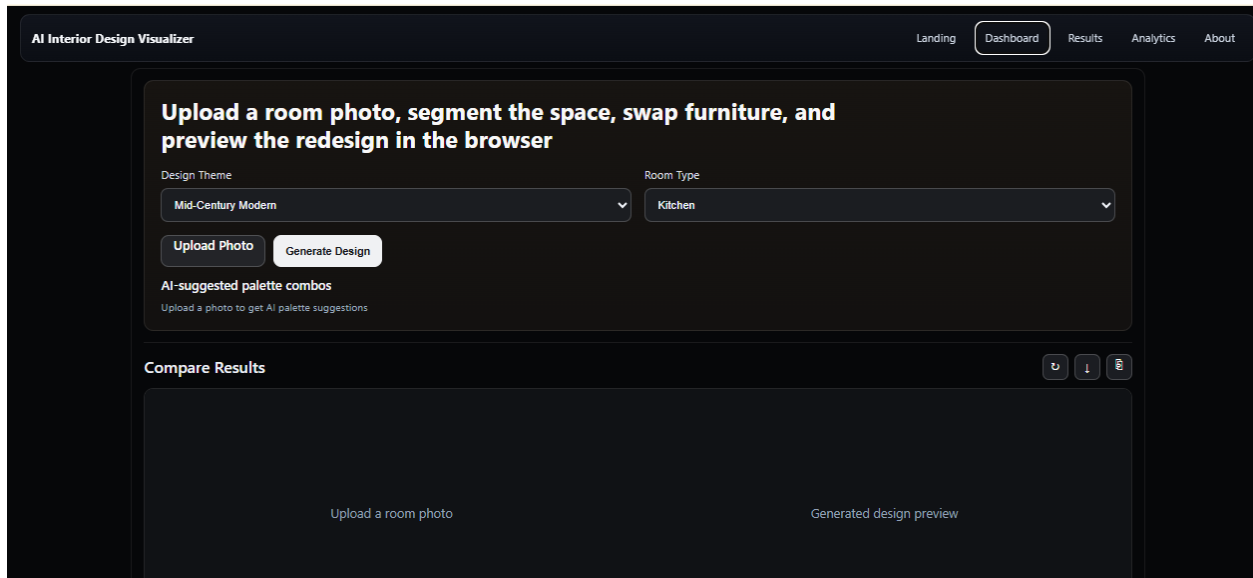
Landing Page

The landing page introduces the project and explains the main idea.



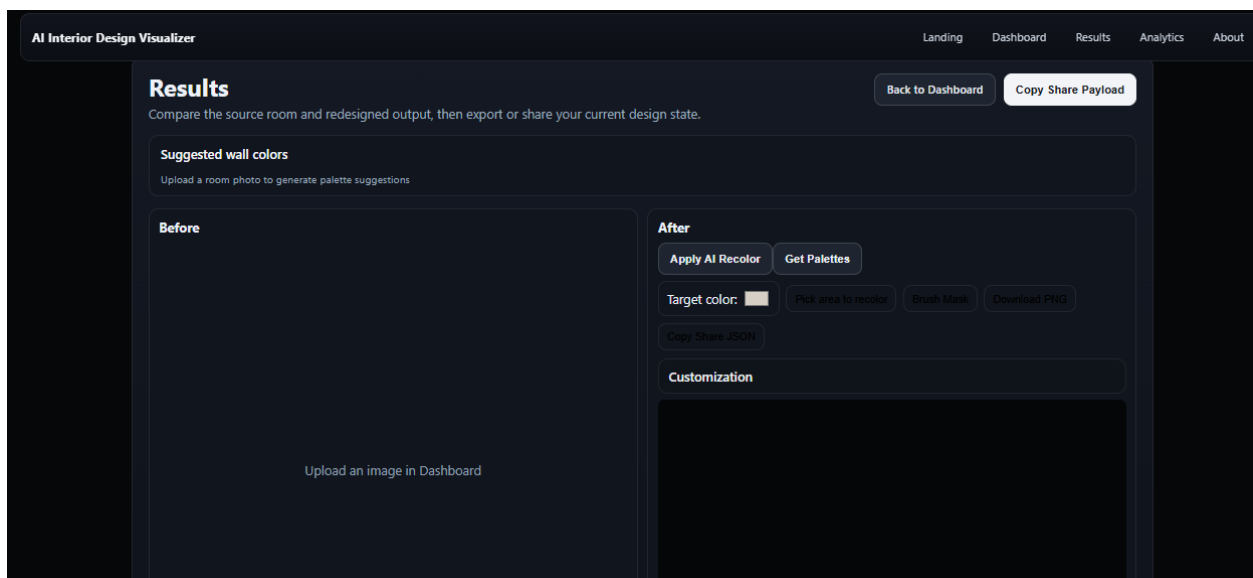
Dashboard Page

The dashboard page allows the user to upload a room image and select design options.



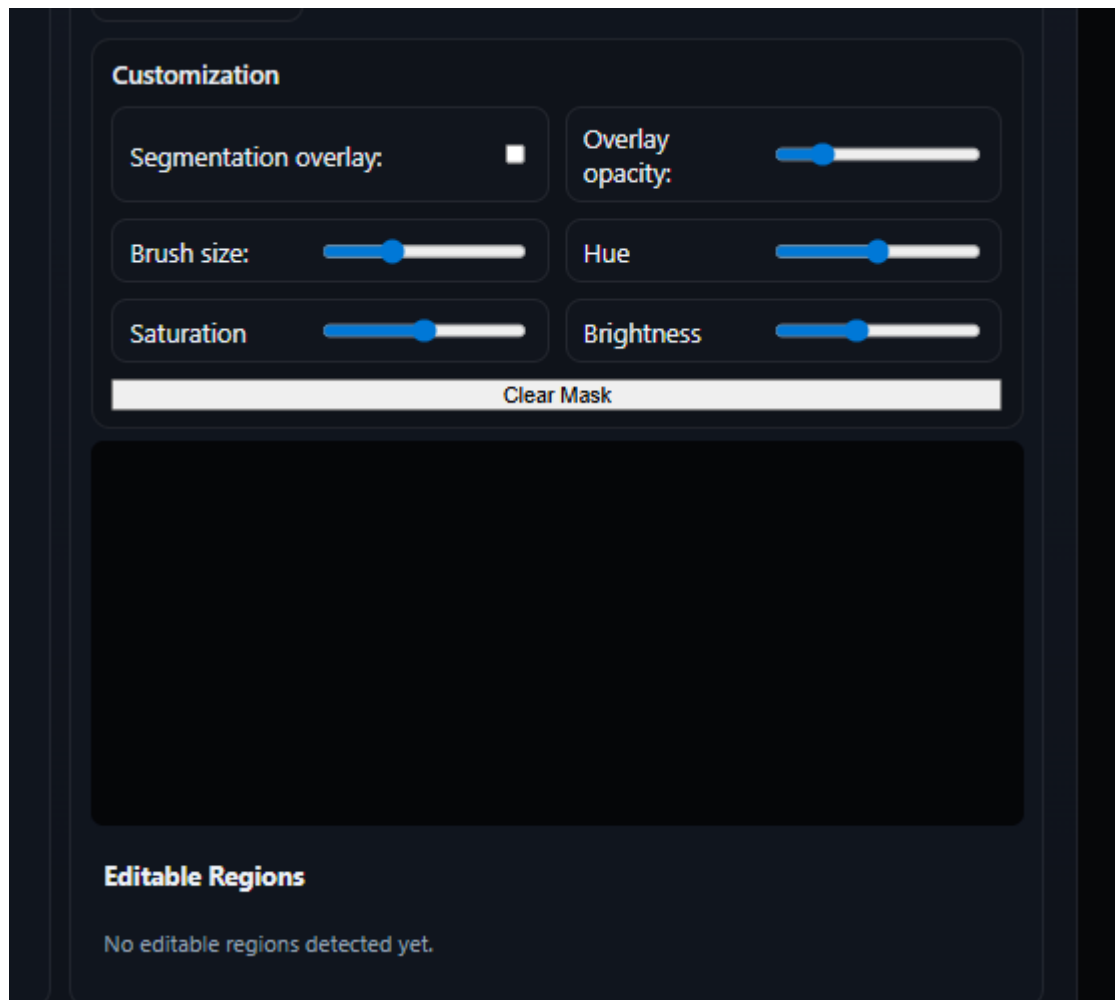
Results Page

The results page shows side-by-side before and after comparison. It also includes the color picker, canvas editor, and design controls.



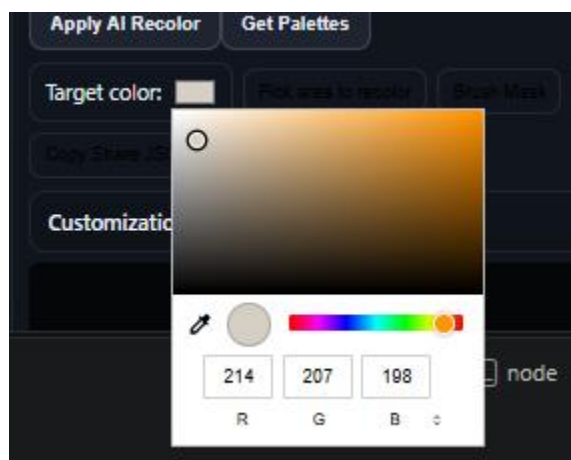
Fabric.js Canvas

Fabric.js is used to display editable design layers. The segmented regions can be shown as canvas layers, and furniture/decor elements can also be placed on the canvas.



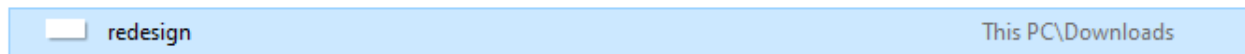
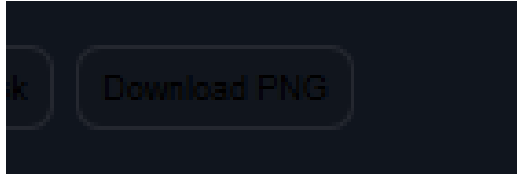
Color Picker

The color picker allows the user to select a wall color. The selected color is sent to the backend and applied to the wall region.



Save and Share Feature

The save/share feature allows the user to save the redesign result and copy a shareable design payload.



15. System Workflow

The complete workflow of the system is as follows:

1. User opens the web application.
2. User uploads a room image.
3. Frontend sends the image to Flask backend.
4. Backend preprocesses the image.
5. AI model segments the room into wall, floor, and furniture regions.
6. Palette engine extracts dominant colors.
7. User selects wall color or design style.
8. Redesign service applies changes to selected regions.
9. Backend saves generated preview image.
10. Frontend displays before/after result.
11. User edits design using canvas.
12. User saves or shares final redesign.

17. Testing and Discussion

The project was tested using different room images. The system was able to upload images, extract palettes, generate segmentation regions, and produce a redesigned preview. The side-by-side comparison helped users clearly see the difference between original and redesigned versions.

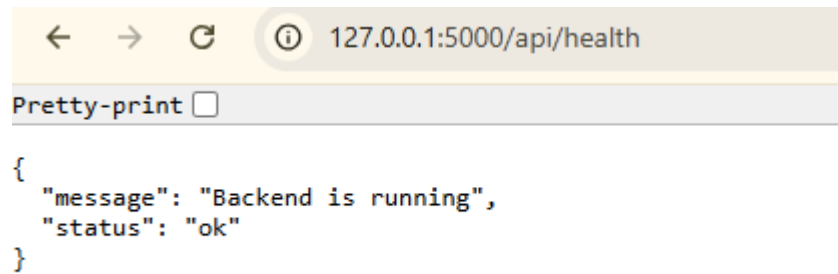
The system worked better on images with clear lighting and visible walls. In images with low light or complex furniture arrangements, segmentation was less accurate. Still, the prototype successfully demonstrated the required AI interior design workflow.

17.1 Functional Testing

Feature	Status	Remarks
Image Upload	Passed	Image uploaded successfully
Palette Extraction	Passed	Color swatches generated
Segmentation	Passed	Wall/floor/furniture masks created
Wall Color Change	Passed	Selected color applied
Before/After View	Passed	Result displayed in browser
Furniture Catalog	Passed	Catalog items available
Save/Share	Passed	JSON design data saved
Flask API	Passed	Backend routes working
React UI	Passed	Frontend pages working

17.2 API Testing

API	Expected Result	Status
/api/health	Backend status OK	Passed
/api/palette	Palette response	Passed
/api/redesign	Preview image URL	Passed
/api/segment	Region masks/layers	Passed
/api/catalog	Furniture items	Passed
/api/designs/save	Saved design ID	Passed



```
(.venv) PS C:\Users\umair\OneDrive\Documents\AI-Interior-Design-Visualizer> curl.exe -i -X POST -F "image=@test_room.jpg"
http://127.0.0.1:5000/api/palette
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Sun, 31 May 2026 18:50:15 GMT
Content-Type: application/json
Content-Length: 2281
Access-Control-Allow-Origin: *
Connection: close

{
  "imageSource": {
    "filename": "20260531185014_test_room.jpg",
    "source": "multipart",
    "url": "/api/uploads/20260531185014_test_room.jpg"
  },
  "palette": [
    "#f0ebe0",
    "#a08365",
    "#78645b",
    "#e6dcd2",
    "#ecdbc9"
  ],
  "recommendations": [
    {
      "designId": "20260531185146",
      "id": "20260531185146",
      "imageUrl": "/api/uploads/20260531185146_test_room.jpg",
      "message": "Redesign generated successfully",
      "payload": {
        "design": {
          "beforeAfter": {
            "afterLabel": "After - japandi",
            "beforeLabel": "Original"
          },
          "catalog": [
            {

```

18. Limitations

Although the project meets the main requirements, there are some limitations:

- Segmentation accuracy depends on image quality.
- Very dark or blurry images may produce weak results.
- Furniture replacement is shown as a style preview and canvas/catalog interaction, not full 3D replacement.
- Processing may be slow on systems without GPU.
- The system works on 2D images only.
- Real-world lighting and shadows can affect color output.
- Some furniture objects may not be perfectly detected.

19. Future Enhancements

In the future, the project can be improved by adding:

- More accurate semantic segmentation models.
- Better furniture object detection.
- More realistic furniture replacement.
- More room styles such as modern, luxury, minimal, and classic.
- Better shadow and lighting correction.
- User login and saved design history.
- Cloud storage for uploaded and generated images.
- Mobile responsive improvements.
- More advanced drag-and-drop furniture editing.

20. Conclusion

The AI Interior Design Visualizer is a useful web-based application that allows users to redesign their rooms digitally before buying anything. The system accepts a room image, performs segmentation, extracts color palettes, applies redesign changes, and displays before/after results in the browser.

The project successfully uses Python, PyTorch-based segmentation, OpenCV, Flask, React.js, and Fabric.js. It includes important features such as room image upload, wall/floor/furniture segmentation, color palette recommendation, color picker, furniture catalog, before/after view, and save/share functionality.

Overall, the project demonstrates how artificial intelligence and web technologies can be combined to create a practical interior design visualization tool.

21. References

1. Meta AI. Segment Anything Model documentation and research resources.
2. OpenCV documentation for image processing and computer vision.
3. Flask documentation for backend API development.
4. React.js documentation for frontend user interface development.

5. Fabric.js documentation for browser-based canvas editing.
6. PyTorch documentation for AI model integration.
